



C PROGRAMMING: BASICS & VARIABLES



Write your first code, understand memory, and master data types.

1 SETUP & COMPILATION ARCHITECTURE

1 CODE EDITOR (VS Code)



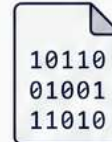
Where you write your English-like C code.

2 COMPILER (GCC/MinGW)



Software containing C rules that checks syntax and translates code.

3 MACHINE CODE



Generates an executable file (.exe for Windows, .out for Linux/Mac) that the computer runs.



PRO TIP

Always add the **MinGW bin** folder to your **Environment Variables** path!

2 ANATOMY OF A C PROGRAM

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World\n");
5
6     return 0;
7 }
```

• **#include**: Preprocessor Directive.

int main(): Execution always starts here.

{ ... }: Curly braces define the block of code.

\n: Next line character.

;: Statement terminator (like a full stop).

return 0: Indicates 0 errors.

1st
Page

3 VARIABLES & MEMORY



A **variable** is the name of a memory location which stores data.

KITCHEN CONTAINER ANALOGY



NAMING RULES (DO'S & DON'TS)

- ✓ Case-sensitive (a and A are different).
- ✓ First character must be an alphabet or underscore (_).
- ✗ No commas or blank spaces.
- ✗ No other special symbols.

4 DATA TYPES & CONSTANTS

DATA TYPES

Data Type	Description	Size (Typical)	Format Specifier
int	Integer values (1, -1, 0)	2 or 4 Bytes	%d
float	Real/Decimal values (3.14)	4 Bytes	%f
char	Single characters	1 Byte	%c

⚠ C does **NOT** have built-in **String** or **boolean** data types!

CONSTANTS

Values that never change.

- **Integer Constants**: 1, 2, 0
- **Real Constants**: 1.0, 3.14
- **Character Constants**: 'A', '*'
(Must be in single quotes!)

5 INPUT, OUTPUT & KEYWORDS

INPUT & OUTPUT (I/O)



OUTPUT: `printf("Age is %d", age);`
– Prints data to the screen.



INPUT: `scanf("%d", &age);`
– Takes user input.



Highlight: The **&** (Ampersand) is the "Address Of" operator, telling the compiler exactly where in memory to store the input.



Remember: Practice small programs daily and build a strong foundation!

KEYWORDS & COMMENTS



KEYWORDS:

32 reserved words in C (e.g., int, return) with fixed meanings. Cannot be used as variable names.



COMMENTS:

Ignored by compiler.

- `//` for single-line comments
- `/* ... */` for multi-line comments



C PROGRAMMING: INSTRUCTIONS & OPERATORS



Master type declarations, arithmetic logic, type conversions, and precedence.

1 TYPES OF INSTRUCTIONS & DECLARATIONS

1 TYPE DECLARATION

```
int a;  
float b;
```

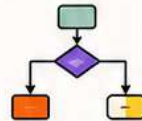
Declaring variables before using them.

2 ARITHMETIC



Performing math (+, -, *, /, %).

3 CONTROL



Deciding the flow (Sequence, Decision if-else, Loop for/while, Case switch).

TYPE DECLARATION RULES (DO'S & DON'TS)

VALID (DO'S)

- ✓ `int a = 22; int b = a;` Declare first, use later.
- ✓ `int a, b, c; a = b = c = 1;` Values assign right-to-left.
- ✓ `int x; int y = x;` Use earlier variable in later statement.

INVALID (DON'TS)

- ✗ `int x = y = z = 4;`
Compiler throws an error because y and z are undefined.
- ✗ `char stars = '***';`
Multiple characters in a single quote throws a warning/error.

2 ARITHMETIC INSTRUCTIONS & QUIRKS

GENERAL RULES



THE LHS RULE

The Left Hand Side (LHS) must ALWAYS be a single variable.

✓ **Correct:** `a = b + c;`

✗ **Incorrect:** `b + c = a;`
(Will throw an error.)

MULTIPLICATION TRAP

You cannot write `bc` or `b.c`. It must explicitly be `b * c`.

✗ `bc` ✗ `b.c` ✓ `b * c`

OPERATOR TRAPS



THE POWER TRAP

Writing `b^c` does NOT calculate power.

It acts as a Bitwise XOR operator and returns 0 for identical numbers (e.g., `8^8` outputs 0).

To calculate power, include `<math.h>` and use `pow(b, c)`.



MODULO OPERATOR (%) DEEP DIVE

Returns the remainder. Works ONLY on integers, never on floats (1.6 % 1.0 is invalid).

SIGN RULE: The remainder takes the sign of the numerator.

`-8 % 3 = -2` `8 % -3 = 2`

3 TYPE CONVERSION

IMPLICIT CONVERSION (AUTOMATIC PROMOTION)

`int OP int` → `int`
Example: `2 / 3` outputs 0, not 0.66

`int OP float` → `float`
Example: `2.0 / 3` outputs 0.666667

`float OP float` → `float`
Example: `1.5 + 2.5` outputs 4.000000

EXPLICIT CONVERSION (MANUAL CASTING)

Example:
`int a = (int) 1.999999;`



KEY TAKEAWAY

C does NOT round off. It completely truncates (drops) the decimals, so a becomes strictly 1.

4 RELATIONAL & LOGICAL OPERATORS



TRUE VS. FALSE IN C

- C has no Boolean keywords.
- 0 = False.
- 1 (or ANY non-zero integer like 2, -5) = True.

RELATIONAL OPERATORS

<code>==</code>	Equal to
<code>></code>	Greater than
<code>>=</code>	Greater than or equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to
<code>!=</code>	Not equal to

⚠ **WARNING:** `==` checks equality, `=` assigns value.

LOGICAL OPERATORS

`&&` (AND)



True ONLY if both are true.

`||` (OR)



True if at least one is true.

`!`



Flips the output.
e.g., `!(5 > 1)` outputs 0 / False.

5 OPERATOR PRECEDENCE & ASSOCIATIVITY

THE PRIORITY LADDER (HIGHEST → LOWEST)

- 1 `!` (NOT)
- 2 `*, /, %` (Multiply, Divide, Modulo)
- 3 `+, -` (Addition, Subtraction)
- 4 `<, <=, >, >=` (Relational)
- 5 `==, !=` (Equality)
- 6 `&&` (Logical AND)
- 7 `||` (Logical OR)
- 8 `=` (Assignment)

ASSOCIATIVITY RULE (LEFT-TO-RIGHT)

When operators share the same priority (like `*` and `/`), solve from **Left to Right**.



EXACT VIDEO EXAMPLE TRACE

Expression: `4 * 3 / 6 * 2`

STEP 1	<code>4 * 3 / 6 * 2</code> (Leftmost *)
	<code>12 / 6 * 2</code>
STEP 2	<code>12 / 6 * 2</code> (Then /)
	<code>2 * 2</code>
STEP 3	<code>2 * 2</code> (Final *)
	<code>4</code>

6 ASSIGNMENT SHORTHAND & PRACTICE PATTERNS

ASSIGNMENT SHORTHAND OPERATORS

Shortcuts to write less, do more.

Operator	Equivalent To	Example
<code>+=</code>	<code>a = a + b</code>	<code>a += b;</code>
<code>-=</code>	<code>a = a - b</code>	<code>a -= b;</code>
<code>*=</code>	<code>a = a * b</code>	<code>a *= b;</code>
<code>/=</code>	<code>a = a / b</code>	<code>a /= b;</code>
<code>%=</code>	<code>a = a % b</code>	<code>a %= b;</code>

Example:
`a += b;`
is exactly
`a = a + b;`

COMMON CODE PATTERNS (FROM THE TUTORIAL PRACTICE)



CHECK IF EVEN

`x % 2 == 0`

Example:
`if (x % 2 == 0) {
 printf("Even");
}`



CHECK IF 2-DIGIT NUMBER

`x > 9 && x < 100`

Example:
`if (x > 9 && x < 100) {
 printf("2-Digit Number");
}`



C PROGRAMMING: CONDITIONAL STATEMENTS



Mastering If-Else, Ternary Operators, Switch Cases, and Logical Traps.

1 REAL-LIFE INTUITION & TYPES



Conditional statements help programs make decisions based on situations.

3rd Page



If marks > 90, grade is A+.



If weather is dark, print "Night".

TWO MAIN TYPES



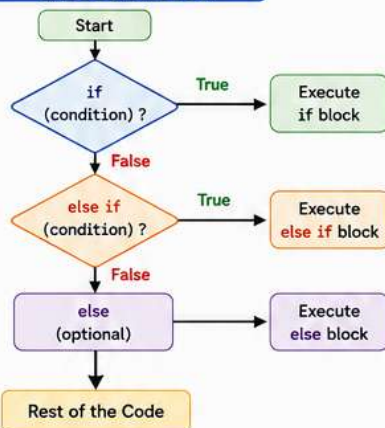
if-else statements



switch statements

2 THE IF-ELSE CONSTRUCT & BRACES RULE

EXECUTION FLOW



- **if:**
Checks the first condition.
- **else if:**
Checks subsequent conditions only if the above fails.
- **else:**
The default fallback if everything above is false.

(Optional to include)

THE CURLY BRACES { } RULE



If you have only **ONE** statement, curly braces are technically **optional**.

Senior Programmer Advice:

ALWAYS
use curly braces!.

If you omit them and write multiple lines, only the first line is considered part of the **if**.

If an **else** follows those unbracketed lines, the compiler will throw an **error** because it loses track of the **matching if**.

3 THE "ASSIGNMENT INSIDE IF" TRAP



TRICKY EXAM / INTERVIEW QUESTION!

```
int x = 2;
if (x = 1) {
    printf("x is equal to 1");
}
else {
    printf("x is not equal to 1");
}
```



THE TRAP

Most people expect this to print "not equal" or throw an error.



THE REALITY

It successfully prints "x is equal to 1"!



THE LOGIC

$x = 1$ is an **assignment**, not an equality check ($==$). The value 1 is assigned to x . Since 1 is a non-zero value, C evaluates it as **True**, forcing the if block to execute.

4 THE TERNARY OPERATOR



Purpose: A one-line shorthand for simple if-else statements.

SYNTAX

Condition ? doSomethingIfTrue : doSomethingIfFalse;



THE SEMICOLON RULE (⚠)

Do **NOT** place a semicolon (statement terminator) right after the **doSomethingIfTrue** part. The statement only ends at the very end of the **entire** ternary expression.

5 THE SWITCH STATEMENT MECHANICS



CONCEPT

Like flipping a physical switch for a light or fan, it executes a specific **case** based on a matching variable.



ALLOWED TYPES

Cases can only check against constants: either Integers (e.g., 1, 2) or Characters (e.g., 'm', 'f').



THE break; TRAP

If you match a case (e.g., case 3:) but forget to write **break;**, ALL subsequent cases will automatically execute until the **switch** ends!



THE default: CASE

Acts like an **else**. Runs if no other case matches. Does not require a **break;**.

```
switch (ch) {
    case 1:
        printf("One\n");
        break;
    case 2:
        printf("Two\n");
        break;
    case 3:
        printf("Three\n");
        break;
    default:
        printf("Invalid Choice\n");
}
// default runs if no case matches
```

EXAMPLE: FORGETTING break;

```
int ch = 3;
switch (ch) {
    case 1:
        printf("One\n");
        break;
    case 2:
        printf("Two\n");
        break;
    case 3:
        printf("Three\n");
        // break; X (forgot!)
    case 4:
        printf("Four\n");
}
// Output:
// Three
// Four (Both printed!)
```

SWITCH PROPERTIES (CHECKLIST)

- ✓ Cases can be in **ANY** order (e.g., case 3 can be written before case 1).
- ✓ Nested switches (a **switch** inside a **switch**) are completely allowed.



C PROGRAMMING: LOOP CONTROL INSTRUCTIONS

Automating repetition: For, While, Do-While, Break, and Continue.



1 THE "WHY" & THE LOOP EQUIVALENCY RULE



THE "WHY"

Loops are used to execute code repeatedly. Instead of copy-pasting

```
printf("Hello World");
```


 10,000 times, a loop automates it in 3 lines.



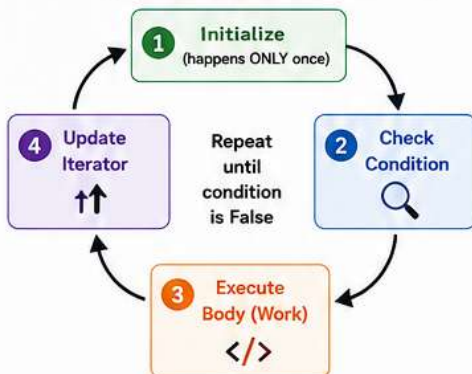
THE EQUIVALENCY RULE

Anything you can do with a **for** loop, you can do with a **while** loop, and you can do with a **do-while** loop! They are completely interchangeable, just with different syntax.

4th Page

2 THE FOR LOOP & ITERATOR TYPES

EXECUTION CYCLE OF FOR LOOP



FOR LOOP SYNTAX

```
for ( initialization; condition; updation ) {
    // work
}
```

initialization:
Executed once before the loop starts.

condition:
Checked before every iteration. If False, loop stops.

updation:
Executed after the body completes, before the condition is checked again.



ITERATOR SUPERPOWER

Loop iterators (usually i, j, k) don't just have to be integers! They can be **float** (e.g., 1.0 to 5.0) or **char** (e.g., 'A' to 'Z').



THE INFINITE LOOP TRAP

If you leave the condition empty or forget to update the iterator, the loop runs forever, crashing the program and filling the computer's memory!

3 INCREMENT & DECREMENT OPERATORS



Concept: Shorthand ways to update iterators.

`i = i + 1` → `i += 1` → `i++`

POST-INCREMENT (i++) / POST-DECREMENT (i--)

Rule: Use, then Increase/Decrease.

Example:

```
printf("%d", i++);
```

Prints the original value, and then internally updates it.

How it works:

`i = 5` → `printf(i++)` → prints 5 → `i becomes 6`



PRE-INCREMENT (++i) / PRE-DECREMENT (--i)

Rule: Increase/Decrease, then Use.

Example:

```
printf("%d", ++i);
```

Internally updates the value first, and then prints the new value.

How it works:

`i = 5` → `++i` → `i becomes 6` → `printf(i)` prints 6

4 WHILE vs. DO-WHILE LOOPS

	WHILE LOOP	DO-WHILE LOOP
Syntax	<pre>initialization; while (condition) { // work; updation; }</pre> Initialization is outside; condition is inside while(); updation is inside the block.	<pre>initialization; do { // work; updation; } while (condition); // Note the semicolon!</pre>
Check	Condition is checked BEFORE any work is done.	The code runs at least ONCE before checking the condition.
When to Use	Use when the loop may execute zero times (if condition is false from the start).	Use when the loop must execute at least once.
Specific Use-Case	—	Perfect for scenarios where you must take user input first before deciding whether to stop. Example: "Keep taking numbers as input until the user enters an odd number".



SEMICOLON TRAP

The do-while loop MUST end with a statement terminator (;) after the while(condition).

5 BREAK vs. CONTINUE



BREAK

ACTION: Completely exits the loop structure.

NESTED TRAP:

If used inside a nested loop, break only exits the INNER loop, not the outer one!

```
for(int i=1; i<=3; i++){
    for(int j=1; j<=3; j++){
        if(j==2) break; // only
        // inner loop breaks
        printf("(%d,%d) ", i, j);
    }
    printf("\n");
}
// Output: (1,1) | (2,1) | (3,1) |
```



CONTINUE

ACTION: Skips the rest of the current iteration and jumps straight to the next updation.

EXAMPLE:

To print numbers 1 to 10 but skip 6, use:

```
for(int i=1; i<=10; i++){
    if(i == 6) {
        continue; // skip 6
    }
    printf("%d ", i);
}
// Output: 1 2 3 4 5 7 8 9 10
```




C PROGRAMMING: FUNCTIONS & RECURSION



Mastering code reusability, arguments, parameters, and recursive logic.

5th
Page

1 THE TV REMOTE ANALOGY & FUNCTION TYPES



THE ANALOGY

A function is a block of code that performs a particular task.



Volume button only changes the volume.



Mute button only mutes the sound.

Similarly, a C function should do exactly one specific job.



THE RULE OF THUMB

If you have to write the same piece of code three or more times, make it a function to increase "Code Reusability".

TWO TYPES OF FUNCTIONS



1 LIBRARY FUNCTIONS

Inbuilt functions provided by C.

Examples:

- `printf()`, `scanf()` → from `<stdio.h>`
- `pow()` → from `<math.h>`



2 USER-DEFINED FUNCTIONS

Functions declared and created by the programmer.

Example:

- `printHello()` → a custom function

2 THE 3-STEP ANATOMY OF A FUNCTION

1



FUNCTION PROTOTYPE (DECLARATION)

Tells the compiler in advance that a function exists.

Syntax:

```
void printHello();  
// void means it returns an empty value / nothing
```

2



FUNCTION DEFINITION

The actual block where the logic is written.

Syntax:

```
void printHello() {  
    printf("Hello");  
}
```

3



FUNCTION CALL

Using the function to do work.

Syntax:

```
printHello();
```



EXECUTION FLOW RULE (IMPORTANT)



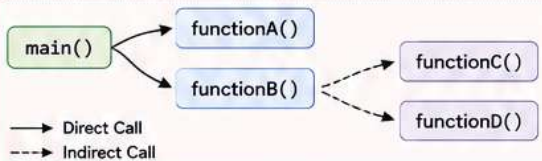
Execution **ALWAYS** starts from the `main()` function.



Even if you write a custom function at the very top of your file, the compiler **IGNORES** it until `main()` explicitly calls it.



Functions can be called directly by `main()` or indirectly by other functions.



3 ARGUMENTS vs. PARAMETERS & THE PASS BY VALUE TRAP

	ARGUMENTS (ACTUAL PARAMETERS)	PARAMETERS (FORMAL PARAMETERS)
Meaning	The values passed inside the function call.	The variables inside the function declaration/definition that catch the values.
Example	<code>sum(a, b)</code>	<code>int sum(int x, int y)</code>
Role	To send the value.	To receive the value.



THE SINGLE RETURN RULE

A function can take 100 parameters, but it can only return **ONE** value at a time.



THE GST PASS-BY-VALUE TRAP



CONCEPT

C passes a copy of the argument, not the original variable.



EXAMPLE

```
int value = 100; (in main)  
calculatePrice(value); // adds 18% GST
```



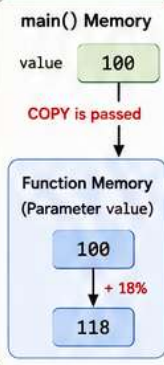
INSIDE FUNCTION

```
Parameter (copy) value = 100  
value = value + 18% → 118  
printf("Price with GST = %d", value); // prints 118
```



BACK IN MAIN()

value is still exactly **100**!
Changes to parameters do NOT reflect in the calling function.



Original in `main()` remains **UNCHANGED**!

4 RECURSION (THE MAGIC CONCEPT)



CONCEPT

Recursion happens when a function calls itself.



MATH ANALOGY

Like calculating $f(f(f(x)))$, where the inner function solves a piece of the puzzle and hands it to the outer function.

TWO MANDATORY PROPERTIES

1 RECURSIVE STEP

How the problem breaks down.

- Sum of first n numbers:
 $sum(n) = sum(n-1) + n$
- Factorial:
 $fact(n) = fact(n-1) * n$

2 BASE CASE

The condition where the function stops calling itself.

```
if (n == 1) return 1;  
// Base case
```

THE EQUIVALENCY RULE

Anything that can be done with iteration (loops) can be done with recursion, and vice versa.

WHY USE RECURSION?

- ✓ Often provides simpler solutions for complex problems.
- ✓ Essential for complex data structures like Trees, Graphs, etc.



5 THE STACK OVERFLOW CRASH



THE TRAP

If you forget to write the Base Case in a recursive function, the function will call itself infinitely.



THE RESULT

The computer's memory fills up completely, causing a **Stack Overflow**, and the program crashes immediately.



THE CRASH

The program throws a **Segmentation Fault** (it can't access memory) and terminates.





C PROGRAMMING: ARRAYS

Mastering contiguous memory, pointer arithmetic, and multidimensional grids.



1 THE DEFINITION & MEMORY LAYOUT



DEFINITION

A collection of similar data types stored at contiguous (continuous) memory locations.



THE "WHY"

Instead of managing 30 separate variables (marks1, marks2, ..., marks30), you create one container that holds all 30 sequentially.

ARRAY OF 3 INTEGERS

Variable Name: arr
Type: int

Memory Address	1000	1004	1008
Index	5	10	15
Value (Data)	0	1	2

Contiguous (Continuous) Memory

MEMORY CALCULATION FORMULA

Total Memory = Number of Elements
× Size of Data Type

Example:

7th Page

Array of 3 integers
= 3 (elements) × 4 bytes (size of int)
= 12 bytes total

2 ZERO-BASED INDEXING & THE "OUT OF BOUNDS" TRAP

ZERO-BASED INDEXING

- ✓ The first item is ALWAYS at index 0.
- ✓ The 3rd item is at index 2.

0	1	2
97	98	89
1st item	2nd item	3rd item

INITIALIZATION SYNTAXES

```
int marks[3] = {97, 98, 89};
```

```
int marks[] = {97, 98, 89};
```

(Compiler automatically detects size as 3)



THE RUN-TIME CRASH TRAP

- If you declare `int marks[3]` and try to assign data to `marks[3]` or `marks[5]`, the compiler will not warn you or show a red line!
- However, when you run it, the program will abruptly abort/crash (Run-Time Error) because you accessed memory that doesn't belong to the array.

```
int marks[3] = {97, 98, 89};
marks[3] = 100; // Invalid! (Out of bounds)
marks[5] = 200; // Invalid! (Out of bounds)
```



CRASH!

3 POINTER ARITHMETIC



ADDITION / SUBTRACTION

`ptr++` does NOT increase the memory address by 1. It increases it by the size of the data type!

Data Type	Current Address	ptr++ Moves To	Bytes Jumped
int* (4 bytes)	2000	2004	+4 bytes
char* (1 byte)	2000	2001	+1 byte



POINTER DIFFERENCES

You can subtract one pointer from another (`ptr2 - ptr1`). The output is NOT the difference in raw bytes, but the difference in the number of elements.

```
int arr[] = {10, 20, 30, 40};
1000 1004 1008 1012 1016
10   20   30   40   50
0     1     2     3     4

int *ptr1 = &arr[1]; // address 1004
int *ptr2 = &arr[3]; // address 1012

ptr2 - ptr1 = 2 // Output is 2 elements
// (Not 8 bytes)
```



STRICT RULE

You cannot subtract pointers of different data types (e.g., an int pointer minus a char pointer throws an error).

```
int *p1; char *p2;
int diff = p1 - p2; // ❌ Error!
```

4 ARRAYS ARE POINTERS!

THE GOLDEN EQUATION

`arr == &arr[0]`



CONCEPT

The name of an array is actually just a pointer pointing to the 0th index of that array.



TRAVERSAL HACK

Because an array is a pointer, you can loop through it using:

```
int arr[] = {10, 20, 30, 40};
int n = 4;
for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
```

```
OR
int *ptr = arr;
int n = 4;
for (int i = 0; i < n; i++)
    printf("%d ", *(ptr + i));
```



Both methods print exactly the same output: 10 20 30 40

5 ARRAYS AS FUNCTION ARGUMENTS

TWO VALID SYNTAXES (Equivalent)

```
// Syntax 1
void printNumbers(int arr[], int n)
{
    // Use arr[i] to access
}

// Syntax 2
void printNumbers(int *arr, int n)
{
    // Use arr[i] to access
    // or *(arr + i)
```

1 IMPLICIT CALL BY REFERENCE



Because arrays decay into pointers, passing an array to a function automatically passes its memory address. Any changes made inside the function (like reversing the array) WILL permanently change the original array in main()!

2 ALWAYS PASS THE SIZE (n)



A function receiving an array has no idea how long it is. You must always pass a second parameter representing the size/length of the array.

6 MULTIDIMENSIONAL ARRAYS (2D ARRAYS)



CONCEPT

Used to store tables of data, like marks for 2 students across 3 subjects.



SYNTAX

```
int marks[2][3];
(2 Rows, 3 Columns)
```

VISUAL REPRESENTATION (As a grid)

	Maths	Physics	Chemistry
Student 1 (Row 0)	90	85	88
Student 2 (Row 1)	78	82	91
	[1][0]	[1][1]	[1][2]

MEMORY LAYOUT (Actual in RAM)

Stored flat & sequentially (Row-major order)

1000	1004	1008	1012	1016	1020
90	85	88	78	82	91
[0][0]	[0][1]	[0][2]	[1][0]	[1][1]	[1][2]

Visual: Matrix/Grid (2D)

Actual: Entirely flat in memory (1D continuous)



FUNCTION PARAMETER RULE

When passing a 2D array to a function, the first dimension size is optional, but you MUST specify the size of the second dimension (columns)!

Example:

```
void printTable(int arr[][10],
               int n, int m);
// 10 = number of columns
```

★ Arrays bring order to memory. Master them, and you master data at scale!



C PROGRAMMING: STRINGS

Mastering character arrays, the Null Character, I/O traps, and <string.h> functions.



1 DEFINITION & THE NULL CHARACTER (\0)



DEFINITION

A String is simply a 1-D character array terminated by a Null Character (\0).

THE "FULL-STOP" ANALOGY



The \0 acts as a **full-stop** for the compiler. Without it, the compiler treats it as a normal character array and doesn't know where the word ends.

MEMORY LAYOUT OF "AMAN"

Index	0	1	2	3	4
Char	'A'	'm'	'a'	'n'	'\0'
ASCII	65	109	97	110	0
Address	2000	2001	2002	2003	2004



The string "Aman" actually occupies 5 memory blocks. The last one is the Null Character (\0) with ASCII value 0.

INITIALIZATION SYNTAXES

1 MANUAL (WITH \0)

```
char name[] = {'A', 'm', 'a', 'n', '\0'};
```

2 AUTOMATIC (DOUBLE QUOTES)

```
char name[] = "Aman";  
// Compiler secretly adds '\0' at the end
```



QUOTE RULE

Characters use **single** quotes ('A'), but Strings **MUST** use double quotes ("Aman").

2 INPUT/OUTPUT & THE "SPACE" TRAP

STANDARD I/O (SINGLE WORDS)

OUTPUT

```
printf("%s", str); // prints string
```

INPUT

```
scanf("%s", str); // reads until space
```



Note: When using `scanf("%s", str);`, you do NOT need the & operator because the array name is already a pointer!

THE MULTI-WORD TRAP

`scanf` cannot take multi-word strings!

If you type:

"Shradha Khapra"

```
scanf("%s", str);
```

Only stores:

"Shradha"

It stops at the first space.

TAKING MULTI-WORD STRINGS

1 gets() / puts()

- `gets(str);` // reads until Enter (\n)
- `puts(str);` // prints with newline



WARNING: `gets()` is dangerous and outdated because it does NOT check array bounds. May cause buffer overflow!

2 fgets() - SAFE & MODERN (RECOMMENDED)

- `fgets(str, n, stdin);`
 - Reads at most `n-1` characters
 - Stops when Enter (\n) is pressed or `n-1` characters are read
 - `stdin` = Standard Input

8th
Page

3 STRINGS AS POINTERS (THE REASSIGNMENT RULE)



Because strings are character arrays, they are inherently pointers. But how you declare them changes what you can do with them later.

POINTER NOTATION



CAN Reinitialize

```
char *str = "Hello World";
```

Later in the code, you CAN change the entire string:

```
str = "Hello"; // Valid
```

ARRAY NOTATION



CANNOT Reinitialize

```
char str[] = "Hello World";
```

Later in the code, `str = "Hello";`

Compiler Error!
(array bases cannot be modified)

4 STANDARD LIBRARY FUNCTIONS (<string.h>)



1 LENGTH: `strlen(str)`

Counts the number of characters. Rule: It explicitly **EXCLUDES** the null character ('\0') from the count.

```
char str[] = "Aman";  
int len = strlen(str); // len = 4  
// 'A', 'm', 'a', 'n' (\0 not counted)
```

3 CONCATENATION: `strcat(firstStr, secStr)`

Appends the second string to the very end of the first string (no spaces added).

```
char firstStr[100] = "Hello";  
char secStr[] = "World";  
strcat(firstStr, secStr);  
// firstStr = "HelloWorld"
```

TRAP: `firstStr` MUST have a large enough capacity to hold the combined result, or the program will crash.



2 COPY: `strcpy(newStr, oldStr)`

Copies the exact value of the old string into the new string, overwriting what was there.

```
char src[] = "Hello";  
char dest[20];  
strcpy(dest, src); // dest = "Hello"
```



4 COMPARE: `strcmp(firstStr, secStr)`

Compares the ASCII values of the strings.

- Returns 0 if perfectly equal.
- Returns a Positive value if `firstStr > secStr`.
- Returns a Negative value if `firstStr < secStr`.

EXACT EXAMPLE: "Apple" vs "Banana"

```
strcmp("Apple", "Banana")  
Compares first characters:  
'A' (65) - 'B' (66) = -1 → Negative  
So, "Apple" < "Banana"
```

5 INDUSTRY PRACTICE: PASSWORD SALTING (REAL-WORLD APPLICATION)

THE CONCEPT

"Salting" is a hacking defense where a company adds a hidden string (salt) to your password before storing it. Even if two users have the same password, their stored passwords will be different.



STRING LOGIC (USING `strcpy` & `strcat`)

```
char password[20], stored[50], salt[] = "123";  
printf("Enter Password: ");  
scanf("%s", password); // user enters "myPass"  
strcpy(stored, password); // copy user password  
strcat(stored, salt); // append hidden salt  
printf("Stored (salted) password: %s", stored);  
// If user entered "myPass" → Output: "myPass123"
```

WHY IT WORKS



An attacker guessing the password won't know the secret salt.



Even if they know the password "myPass", they can't match "myPass123".



Every user gets the same password but stored differently with the salt.

★ **KEY TAKEAWAY:** Treat every string like a character array with a mandatory "\0" at the end. | Use safe I/O, understand pointers, and master <string.h>! 🚀



C PROGRAMMING: STRUCTURES

Grouping diverse data, the Arrow Operator, Typedef Aliases, and Custom Data Types.



1 THE "WHY" & USER-DEFINED DATA TYPES

ARRAYS vs STRUCTURES

ARRAYS

Store collections of the SAME data type.

Example:

```
int marks[3] = {95, 90, 88};
```

95	90	88
0	1	2

VS

STRUCTURES (struct)

Store collections of DIFFERENT data types under one entity.

Example: Student

Name	: Shradha
Roll No.	: 1664
CGPA	: 9.20



USER-DEFINED DATA TYPE

While int and float are built-in types, a Structure is a **USER-DEFINED DATA TYPE**. You are teaching the compiler a brand-new data format!

THE SYNTAX & SEMICOLON TRAP

SYNTAX:

```
struct student {
    int roll;
    float cgpa;
    char name[100];
};
```

9th Page

← MANDATORY SEMICOLON!

TRAP:

Students constantly forget the Statement Terminator (;) at the very end of the closing curly brace. It is **ABSOLUTELY MANDATORY!**

2 INITIALIZATION, DOT OPERATOR & THE STRING TRAP



PIECEWISE INITIALIZATION (Using Dot Operator)

```
struct student s1;
s1.roll = 1664;
s1.cgpa = 9.2;
strcpy(s1.name, "Shradha");
```

Use the DOT OPERATOR (.) to access structure members.

Example: s1.roll = 1664;



THE STRING ASSIGNMENT TRAP

You **CANNOT** write s1.name = "Shradha"; because name is declared using array notation, which cannot be reassigned.

```
s1.name = "Shradha"; // ERROR!
```

You **MUST** use <string.h> and strcpy().

```
strcpy(s1.name, "Shradha");
```

SINGLE-LINE INITIALIZATION

```
struct student s1 = {1664, 9.2, "Shradha"};
```



Values **MUST** be in the exact order of declaration (roll, cgpa, name).



ZEROING TRICK

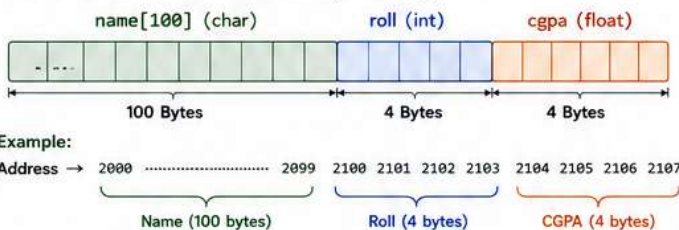
```
struct student s1 = {0};
```

Automatically initializes all properties to null / zero.

3 MEMORY LAYOUT & ARRAY OF STRUCTURES

CONTIGUOUS MEMORY LAYOUT

Structures store their data continuously in memory.



ARRAY OF STRUCTURES

Instead of making 100 separate variables, create an array of your custom type!

```
struct student ece[100]; // Array of 100 students
```

ACCESSING MEMBERS

```
ece[0].roll = 1664; // 0th student's roll number
ece[i].cgpa = 9.5; // i-th student's CGPA
```

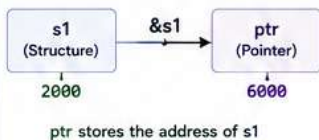


Think: First access the specific student using [index], then use the dot operator (.) to access their member.

4 POINTERS TO STRUCTURES & THE ARROW OPERATOR (->)

POINTER SYNTAX

```
struct student *ptr = &s1;
```



THE PARENTHESIS PROBLEM

If you try to access a member via pointer:

```
*ptr.roll // ERROR!
```

Why? Because * has higher precedence than .

Correct (But ugly):

```
(*ptr).roll // Works, but confusing!
```



THE ARROW OPERATOR SOLUTION

C provides the Arrow Operator (->) to simplify this.

```
ptr->roll // Clean, simple & standard!
```

How -> works internally:



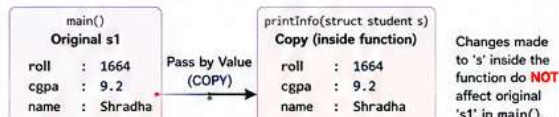
Use -> whenever you access structure members via a pointer.

5 FUNCTION PASSING & THE typedef ALIAS



PASSING STRUCTURES TO FUNCTIONS

When you pass a structure to a function (e.g., printInfo(s1)), C passes it by **VALUE** (as a copy).



If you want changes to reflect back, pass an address (pointer)!



THE typedef KEYWORD (NICKNAME / ALIAS ANALOGY)

Just like James Bond goes by the alias "007", or a friend Suresh goes by the nickname "Chintu", typedef gives long structure names a catchy nickname!

WITHOUT typedef (Long & Repetitive)

```
struct ComputerEngineeringStudent {
    int roll;
    float cgpa;
    char name[100];
};
struct ComputerEngineeringStudent s1;
```



Typing the full name every time is exhausting!

WITH typedef (Short & Sweet)

```
typedef struct ComputerEngineeringStudent {
    int roll;
    float cgpa;
    char name[100];
} coe; // 'coe' is the alias!
coe s1; // Much better!
```



Clean, simple & easy to use!

★ **KEY TAKEAWAY:** Structures help you model real-world entities. Use them smartly with pointers, arrays, and functions!



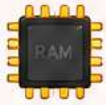
C PROGRAMMING: FILE I/O

Mastering persistent data, reading/writing files, EOF logic, and File Pointers.



1 THE "WHY" & VOLATILITY

THE PROBLEM: VOLATILE (LOST!)



RAM

(Random Access Memory)

Normal variables in C live in RAM.

It is **Volatile**.

When the program ends or power fails, the data is **permanently lost**.

THE SOLUTION: NON-VOLATILE (SAVED!)



HARD DISK

(Non-Volatile Storage)

Files are stored on Non-Volatile storage devices (like Hard Disk).
File I/O (Input/Output) allows C to talk to these files to store and retrieve data long-term.



KEY TAKEAWAY

Use Files to persist data beyond the life of a program.



2 FILE TYPES & THE FILE POINTER

TWO TYPES OF FILES



TEXT FILES

Easily readable by humans.

Examples:
.txt, .c, .log



BINARY FILES

Stored as 0s and 1s, used by computer systems.

Examples:
.jpg, .dat, .exe

THE FILE POINTER (FILE *fptr)

FILE *fptr



- FILE is a hidden internal structure in C.
- We must create a pointer to this structure to establish a communication line between the C program and the file.

SYNTAX

```
FILE *fptr;  
// Declare file pointer
```

3 OPENING, CLOSING & FILE MODES

1

OPEN

Open a file using
fopen()

2

WORK

Read from /
Write to file

3

CLOSE

Close file using
fclose()

Opening a File : `fptr = fopen("filename.txt", "mode");`

Closing a File : `fclose(fptr);` // Crucial to release system resources!

FILE MODES	DESCRIPTION	BEHAVIOR
"r"	Open for Reading.	If file doesn't exist, fopen returns NULL.
"rb"	Open for Reading in Binary.	Same as "r", but for binary files.
"w"	Open for Writing.	If file doesn't exist, it is created. If it exists, contents are completely overwritten.
"wb"	Open for Writing in Binary.	Same as "w", but for binary files.
"a"	Open for Append.	If file doesn't exist, it is created. If it exists, new data is added to the end.



SAFETY CHECK PRACTICE: Always check if `fptr == NULL` before reading/writing!

```
if (fptr == NULL) { printf("Error opening file!"); return 1; }
```

4 READING AND WRITING DATA

READING (fscanf)



- Works exactly like scanf, but requires the file pointer as the first argument.

EXAMPLE

```
int num;  
fscanf(fptr, "%d", &num);  
// Reads an integer from file
```

WRITING (fprintf)



- Works exactly like printf, but requires the file pointer as the first argument.

EXAMPLE

```
int num = 432;  
fprintf(fptr, "%d", num);  
// Writes integer to file
```

10th Page

5 CHARACTER I/O & THE EOF TRAP

CHARACTER I/O FUNCTIONS



`char fgetc(FILE *fptr)`
Reads a single character from the file.



`int fputc(int ch, FILE *fptr)`
Writes a single character to the file.



THE EOF (END OF FILE) LOOP

- When reading a file character-by-character, you must stop when the file ends.
- fgetc returns a special macro called EOF when there is nothing left to read.

WHAT IS EOF?

EOF (End Of File) is a macro constant (usually -1) defined in `<stdio.h>`. It is returned by `fgetc()` to signal that the end of the file has been reached.

STANDARD CODE PATTERN

```
1 char ch;  
2 while(1) {  
3     ch = fgetc(fptr);  
4     if(ch == EOF) {  
5         break; // End of File  
6     }  
7     printf("%c", ch); // Process character  
8 }  
  
// This loop reads and prints the entire file character-by-character.
```

★ **REMEMBER:** Always open the file, do your work, and close the file. Good file handling = Good programs!



C PROGRAMMING: DYNAMIC MEMORY ALLOCATION (DMA)

Mastering Run-Time memory, `<stdlib.h>`, `malloc`, `calloc`, `realloc`, and `free`.



1 THE "WHY" (COMPILE-TIME vs. RUN-TIME)



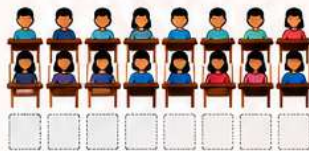
STATIC MEMORY (COMPILE-TIME)

When you declare an array like

```
int arr[30];
```

the compiler fixes the size permanently before the program even runs.

CLASSROOM (30 SEATS FIXED)



VS



DYNAMIC MEMORY (RUN-TIME)

DMA allows you to request, expand, or shrink memory while the code is executing (Run-Time), solving the rigid array problem.

CLASSROOM (SEATS ADJUST AT RUN-TIME)



ADAPTIVE SIZE



THE PROBLEM

3 NEW STUDENTS JOIN MID-YEAR



You can't expand the array to 33.

ONLY 10 STUDENTS SHOW UP



20 spaces are wasted!



REQUIREMENT

You **MUST** include the standard library

```
#include <stdlib.h>
```

to use Dynamic Memory Allocation functions.

11th Page

2 THE sizeof() OPERATOR & VOID POINTERS



sizeof() FUNCTION

Memory is calculated in bytes, but an integer's byte size can vary by machine (e.g., 2 bytes or 4 bytes).

`sizeof(int)` safely checks your specific system's architecture and returns the exact bytes needed.

EXAMPLE



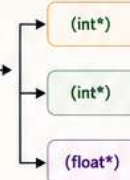
On one system:
`sizeof(int) = 2` bytes

On another system:
`sizeof(int) = 4` bytes



THE TYPECASTING RULE

DMA functions return a `void*` (a generic pointer with no data type). You **MUST** explicitly typecast it to the type you need.



3 malloc vs. calloc (THE MEMORY CREATORS)



malloc (MEMORY ALLOCATION)

SYNTAX

```
ptr = (int*) malloc(5 * sizeof(int));
```

ARGUMENT

Takes **ONE** argument
(Total number of bytes).

DATA STATE

Leaves the reserved memory blocks completely uninitialized.
(Contains random Garbage Values)

MEMORY BLOCKS (Example)



calloc (CONTIGUOUS ALLOCATION)

SYNTAX

```
ptr = (int*) calloc(5, sizeof(int));
```

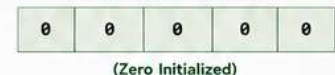
ARGUMENTS

Takes **TWO** arguments
(Number of locations, Size of each location).

DATA STATE

Strictly initializes every single block to 0 (or 0.0 for floats) by default.

MEMORY BLOCKS (Example)



THE NULL TRAP

If you request absurdly large memory (e.g., 5 Crore bytes) and the computer's RAM cannot provide it, `malloc` and `calloc` will return `NULL`.

Check Like This!

```

if(ptr == NULL) {
    printf("Memory not allocated!");
    exit(1);
}
  
```

4 realloc (THE MEMORY RESIZER)



CONCEPT (RE-ALLOCATION): If you initially allocated memory for 5 numbers, but suddenly need to store 8 numbers, `realloc` safely expands the same pointer's capacity without losing the old data. It can also shrink memory to save resources.

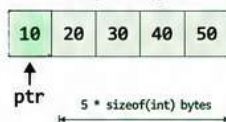
SYNTAX

```
ptr = realloc(ptr, 8 * sizeof(int));
```

1 PARAMETER 1: The original pointer

2 PARAMETER 2: The new total size

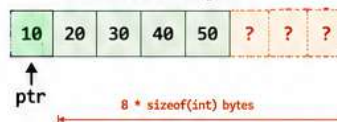
Initially: 5 Integers



EXPAND

(Realloc)

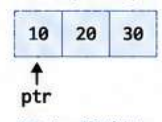
Now need 8 Integers



SHRINK

(Realloc)

Now only 3 Integers



★ Old data is preserved up to the minimum of old size and new size.

5 free & MANUAL MEMORY MANAGEMENT



DON'T LET MEMORY GO TO WASTE!



THE PROBLEM

Normal variables (static memory) are automatically cleared by the C compiler when they go out of scope or the program ends. However, the compiler is entirely blind to Dynamic Memory.



THE RULE

It is strictly the programmer's responsibility to release dynamic memory once it's no longer needed.



SYNTAX

```
free(ptr);
```

// Releases the memory pointed by ptr



WARNING

Failing to free memory means those RAM resources remain blocked and occupied.

- ✗ Leads to inefficient code
- ✗ Wastes precious RAM
- ✗ Can cause eventual system crashes!



MEMORY LEAK = DANGER!



KEY TAKEAWAY

DMA gives you flexibility and power, but with great power comes great responsibility.
ALWAYS allocate smartly and free responsibly!

// Good Programmer
Allocate → Use → Free



C PROGRAMMING: ULTIMATE CHEAT SHEET

Core mechanics, golden rules, and top interview questions for quick revision.



1 THE GOLDEN RULES OF C ⚠️

Aa 1. C is **CASE-SENSITIVE**.
A and a are completely different memory locations.

T/F 2. **TRUE vs FALSE**: C has no boolean data types.
0 is exactly False, and **ANY** non-zero value (e.g., 1, -5) evaluates to **True**.

🚩 3. **THE EXECUTION ENTRY POINT**:
The compiler ignores everything until it finds `int main()` — execution **ALWAYS** starts there.

! 4. **STATEMENT TERMINATOR**:
Almost every line of action must end with a **semicolon (;)**.
Missing this is the #1 cause of beginner compilation errors.

2 MEMORY & POINTERS SUMMARY

MEMORY MAP (VISUAL)

Address	Low	1000	1004	1008	1012	1016	High
int arr[5]		10	20	30	40	50	
		0	1	2	3	4	
		Index					

THE ARRAY = POINTER LAW
An array's name is a constant pointer to its 0th index.

```
arr == &arr[0]
```

OPERATORS REFRESHER

& (AMPERSAND):
"Tell me your memory address!"

***** (ASTERISK):
"Tell me the value inside this memory address!"

CALL BY VALUE vs CALL BY REFERENCE

VALUE: Passing a copy. Original is safe. → No Change to Original

REFERENCE: Passing the &address. Original is permanently mutated. → Original Changes!

★ Always use Call by Reference for swapping or mutating arrays.

3 ESSENTIAL SYNTAX FORMATS

1 TERNARY OPERATOR

```
condition ? true_val : false_val;
```

Example:
`max = (a > b) ? a : b;`

2 STRING INITIALIZATION

```
char str[] = "Hello";  
// Compiler auto-adds '\0'
```

'H'	'e'	'l'	'l'	'o'	'\0'
0	1	2	3	4	5

TYPEDEF STRUCT

```
typedef struct {  
    int a;  
} aliasName;
```

Example:
`aliasName obj1; // Use alias!`

DYNAMIC MEMORY (MALLOC)

```
ptr = (int*) malloc(n * sizeof(int));
```

n = number of elements
sizeof(int) = size of one element (in bytes)
Total bytes = n * sizeof(int)

4 TOP INTERVIEW / EXAM QUESTIONS

Q1 What happens if you forget the base case in recursion?

💡 The function calls itself infinitely, exhausting computer memory. This causes a **"Stack Overflow"** and results in 1 Segmentation Fault / Program Crash.

Q2 What is the difference between malloc and calloc?

💡 **malloc** leaves the requested memory uninitialized (filled with garbage values).
calloc strictly initializes every single bit of the requested memory to 0.

Q3 Why must you use free()??

💡 Dynamic memory is invisible to the compiler's automatic cleanup. If you don't **free(ptr)**, the memory stays occupied forever, causing a **Memory Leak**.

Q4 What is the "Fall-through" effect in a Switch statement?

💡 If you forget the **break;** keyword inside a case, C will blindly execute all subsequent cases until the switch ends.

5 END OF JOURNEY




C is a foundational, low-level language. It doesn't hold your hand, which forces you to understand exactly how RAM, CPUs, and Data Structures operate. Once you master C, transitioning to C++, Java, or Python will be incredibly fast and intuitive. **Keep practicing!** 🙌🚀

